

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

CE-1113 Sistemas Empotrados

Documento de Diseño (DI)

Proyecto I — Sistema embebido a la medida para un robot aspiradora autónomo con reproducción de audio y control remoto

Integrantes

[PENDIENTE: nombres y carnés de los cuatro integrantes]

Profesor

Dr.-Ing. Jeferson González Gómez

Octubre de 2026

Índice

1. Introducción	2
2. DI1 — Necesidades y requerimientos	2
2.1. El problema	2
2.2. Partes interesadas y sus necesidades	2
2.3. Requerimientos técnicos	3
2.4. Salud y seguridad pública	4
2.5. Costos	4
2.6. Impacto ambiental	4
2.7. Recursos disponibles y restricciones	5
3. DI2 — Valoración de alternativas	5
3.1. Método	5
3.2. Detección de obstáculos	6
3.3. Alimentación	6
3.4. Salida de audio	6
3.5. Odometría	7
3.6. Otras decisiones	7
4. DI3 — Diseño de la alternativa seleccionada	7
4.1. Arquitectura de hardware	7
4.2. Arquitectura de software	8
4.3. Navegación	9
4.4. Elementos propios del diseño	10
4.5. Cómo responde el diseño a seguridad, costo e impacto	10
5. DI4 — Validación del diseño final	10
5.1. Estrategia	10
5.2. Matriz de cumplimiento	11
5.3. Validación de la seguridad	12
5.4. Validación de la eficiencia	13
5.5. Validación del costo y del impacto	13
5.6. Hallazgos de la validación y acciones correctivas	13
5.7. Limitaciones conocidas	14
6. Conclusiones	14
7. Referencias	14

1. Introducción

Este documento evidencia el atributo de **Diseño (DI)** en el Proyecto I del curso. Recorre el proceso completo: qué se necesitaba y bajo qué restricciones (DI1), qué alternativas se valoraron y con qué criterios (DI2), cómo quedó diseñada la alternativa elegida (DI3) y cómo se comprobó que el resultado cumple los requerimientos y las consideraciones de seguridad, costo e impacto (DI4).

El producto es un robot aspiradora de tracción diferencial sobre una Raspberry Pi 4 con una imagen de Linux mínima construida con Yocto (Poky, rama `scarthgap`). Navega de forma autónoma evadiendo obstáculos, reproduce MP3 y se opera desde un panel web que muestra la telemetría y el mapa de recorrido en tiempo real. Todo acceso al hardware pasa por una biblioteca dinámica propia, `librobot.so`, compilada de forma cruzada.

El detalle técnico —cálculos, listas de materiales, procedimientos de verificación y evidencias— está en la carpeta `docs/` del repositorio. Aquí se resume y se cita el archivo correspondiente.

2. DI1 — Necesidades y requerimientos

2.1. El problema

Un robot de limpieza coordina en tiempo real subsistemas que compiten por los mismos recursos: sensado de proximidad, tracción, audio, red e interfaz de usuario. El problema es complejo por tres razones que no se pueden resolver por separado:

- **Recursos acotados.** El enunciado fija un presupuesto de 200 MB de sistema de archivos raíz y 15 s de arranque. Los GPIO entregan 16 mA por pin y 50 mA en total, y no toleran más de 3.3 V.
- **Dominios eléctricos incompatibles.** La lógica trabaja a 3.3 V y miliamperios; los motores, a 7.4 V con picos de varias veces su corriente nominal. Deben convivir sin compartir tierra.
- **Concurrencia.** Navegar, medir, reproducir audio y atender la interfaz ocurren a la vez, y ninguna tarea puede degradar la respuesta ante un obstáculo.

2.2. Partes interesadas y sus necesidades

Parte interesada	Necesidad
Usuario final	Un robot que limpie sin supervisión, que se pueda tomar a mano desde el celular y cuyo estado se entienda de un vistazo (LEDs, sonidos, mapa).
Curso y profesor	Cumplimiento verificable del enunciado, y que el equipo prestado (Raspberry Pi 4 y su kit) regrese sin daño.
Grupo de trabajo	Un diseño que cuatro personas puedan construir en seis semanas, con el inventario disponible y un costo que el grupo pueda asumir.
Terceros en el entorno	Que el robot no represente un riesgo: ni eléctrico, ni de incendio, ni por golpear a personas u objetos, ni por exponer la red.

2.3. Requerimientos técnicos

Los requerimientos obligatorios del enunciado se numeran para trazarlos después en la validación (sección 5.2).

Id.	Área	Requerimiento
R1	Navegación	Modo autónomo con al menos un algoritmo de cobertura reactiva. Ante un obstáculo: detenerse, retroceder y cambiar de dirección.
R2	Sensado	Al menos dos sensores de proximidad para detección frontal y lateral, leídos y procesados en tiempo real.
R3	Tracción	Dos motores DC con control diferencial (avance, retroceso, giros, detención) y velocidad por PWM para giros de radio variable.
R4	Indicadores	Cuatro LEDs: modo autónomo, modo manual, obstáculo y sistema encendido.
R5	Audio	Reproducir MP3 locales de forma concurrente con la navegación.
R6	Audio	Desde la interfaz: elegir canción, reproducir, pausar, detener y ajustar el volumen.
R7	Audio	Salida directa desde el robot, con parlante a bordo.
R8	Audio	Sonidos de notificación en cuatro eventos: inicio del sistema, inicio del modo autónomo, obstáculo y cambio a manual.
R9	Control remoto	Conmutar entre modo autónomo y manual; en manual, los controles direccionales mandan los motores.
R10	Control remoto	Panel con modo activo, controles, sensores en tiempo real, control de audio, estado de los LEDs y mapa.
R11	Mapa	Grilla 2D incremental (visitada, obstáculo, desconocida) a partir de los sensores y la odometría, visible en tiempo real.
R12	Seguridad	Pantalla de inicio de sesión con al menos un usuario y un protocolo de seguridad mínimo.
R13	Arranque	Inicio automático con una unidad <code>systemd</code> propia, <code>Restart=on-failure</code> y sin interfaz gráfica local.
R14	Biblioteca	Biblioteca dinámica propia que encapsule motores, sensores, LEDs y audio; el servidor la usa de forma exclusiva.
R15	Desarrollo	Todo el software se compila de forma cruzada en el host.
R16	Sistema operativo	Imagen mínima con Yocto, solo con los paquetes necesarios y cada uno justificado.
R17	Construcción	CMake o Autotools, generando el paquete estándar.
R18	Receta	Al menos una receta <code>.bb</code> propia; la imagen se reproduce con <code>bitbake <imagen></code> sin pasos manuales.
R19	Eficiencia	Medir y reportar rootfs, tiempo de arranque, RAM y CPU en operación normal. Referencia: rootfs ≤ 200 MB y arranque ≤ 15 s.

Los tres requerimientos opcionales (detección de desnivel, notificación de fin de ciclo y playlist persistente) se dejaron fuera del alcance para concentrar el tiempo en el núcleo obligatorio.

2.4. Salud y seguridad pública

Riesgo	Requerimiento que se deriva
Incendio o fuga térmica de las celdas de Li-Ion	BMS con protección de sobrecarga, sobredescarga, cortocircuito y balanceo; fusible en serie con el pack; nunca alimentar la Raspberry directo de la celda.
Daño al equipo prestado	Aislamiento óptico entre la lógica y la etapa de potencia, con tierras separadas; ninguna señal de 5 V llega a un GPIO.
Golpes a personas u objetos	Freno automático ante un obstáculo también en modo manual; masa menor a 1.5 kg; los motores quedan frenados en reposo y durante el arranque.
Cables en las llantas	Ruteo fijo del cableado, sin tramos sueltos a menos de 1 cm de una llanta.
Acceso no autorizado por la red	Inicio de sesión obligatorio; el demonio de GPIO solo escucha en la interfaz local; sin acceso remoto sin contraseña en la imagen de entrega.
Nivel sonoro	Volumen ajustable desde la interfaz y potencia máxima acotada por el divisor de entrada del amplificador.

2.5. Costos

La Raspberry Pi 4 y su kit los presta el curso, de modo que el costo para el grupo es el del modelo físico. La tabla es la estimación con que se planeó la compra, por bloques.

Bloque	Estimación (USD)
Alimentación portátil (pack 2S, BMS, reguladores, fusible, interruptor)	25–45
Circuitería y sensores (optoacopladores, servo, pasivos, placas)	9–17
Etapa de audio (PAM8403, parlante, filtro)	5–12
Chasis y acabado	10–25
Total del robot	49–99

[PENDIENTE: sustituir la estimación por el costo real, con las facturas de la compra]

El requerimiento que se fijó fue mantenerse dentro de ese intervalo y aprovechar primero lo que el grupo ya tenía: un sensor ultrasónico, un MPU-6050 y un driver L298N.

2.6. Impacto ambiental

- **Baterías.** Las celdas de Li-Ion son el componente de mayor impacto. Se exige que sean recargables y desmontables, y que al final de su vida se entreguen a un gestor de residuos electrónicos.

- **Residuos electrónicos.** El montaje debe permitir reutilizar los módulos: conectores desconectables y nada pegado de forma permanente a la Raspberry Pi, que se devuelve al curso.
- **Consumo.** Menos consumo es más autonomía con el mismo pack y menos ciclos de carga: se prefieren reguladores conmutados y amplificación clase D.
- **Compras.** Reutilizar el inventario del grupo antes que comprar componentes nuevos.

2.7. Recursos disponibles y restricciones

Recurso	Disponibilidad
Plataforma	Raspberry Pi 4 y kit, prestados por el curso.
Inventario del grupo	Un HC-SR04, un MPU-6050 (módulo GY-521) y un driver L298N de doble puente.
Personas y tiempo	Cuatro integrantes, del 25 de agosto al 6 de octubre de 2026.
Host de construcción	Laptop de 4 núcleos y 7 GB de RAM, por debajo de lo recomendado para Yocto.
Software de partida	Un proyecto de referencia con licencia MIT, del que derivan parte de la capa y del código (atribuido en <code>NOTICE.md</code>).
Herramientas	Multímetro, cautín, y el Toolchain-SDK generado por el propio proyecto.

De ahí salen dos restricciones que marcaron el diseño: el hardware llegó tarde respecto del software, así que hacía falta poder desarrollar y probar sin él; y el host de construcción era el cuello de botella de cada iteración de la imagen.

3. DI2 — Valoración de alternativas

3.1. Método

Las decisiones de mayor peso se evaluaron con una matriz ponderada. Cada alternativa recibe una nota de 1 (peor) a 5 (mejor) en cuatro factores:

Factor	Peso	Qué mide
Técnico	40 %	Cumplimiento del requerimiento, desempeño, uso de GPIO y riesgo de integración.
Económico	25 %	Costo de compra y disponibilidad local; cuánto del inventario se aprovecha.
Ambiental	15 %	Cantidad de componentes, consumo y reutilización.
Social	20 %	Seguridad de las personas y del equipo prestado.

El factor técnico pesa más porque una alternativa que no cumple el requerimiento no es una alternativa. El social pesa más que el ambiental porque el enunciado hace al grupo responsable del equipo prestado.

3.2. Detección de obstáculos

Alternativa	Téc.	Econ.	Amb.	Soc.	Total
Tres HC-SR04 fijos	3	3	3	4	3.20
Un HC-SR04 sobre un servo (radar)	4	4	4	3	3.80
Sensores infrarrojos	2	4	3	2	2.65

Los tres sensores fijos dan una lectura frontal rápida (≈ 16 Hz), pero solo tres direcciones, ocupan seis GPIO, obligan a secuenciar los disparos para evitar ecos cruzados y exigían comprar dos sensores más. Los infrarrojos económicos tienen poco alcance, dependen del color de la superficie y de la luz ambiente, y no entregan una distancia útil para el mapa.

El radar mide siete direcciones con tres GPIO y el sensor que el grupo ya tenía. Su debilidad es que mira al frente una vez cada 1.2 s; por eso su nota social es menor. Esa debilidad se atacó en el diseño con la estimación del tiempo antes de chocar (sección 4.4).

3.3. Alimentación

Alternativa	Téc.	Econ.	Amb.	Soc.	Total
Una celda (1S) con elevador a 5 V	2	4	4	2	2.80
Pack 2S con BMS y reductores	5	3	3	5	4.20
Powerbank USB y batería aparte para motores	3	3	2	4	3.05

Con una celda, entregar 15 W a 5 V pide cerca de 4.5 A de entrada, y más de 5 A con la celda descargada: los elevadores MT3608 y XL6009 no lo sostienen, y una caída de tensión corrompe la microSD. El powerbank resuelve la lógica pero obliga a un segundo pack para los motores y deja de ser una sola fuente regulada por separado. Con 2S, un reductor trabaja con holgura en todo el rango de descarga (6.0–8.4 V) y los motores se alimentan directo.

3.4. Salida de audio

Alternativa	Téc.	Econ.	Amb.	Soc.	Total
Jack de 3.5 mm con LM386	3	4	3	3	3.25
PWM por GPIO 18 con PAM8403	4	5	4	4	4.25
DAC I2S (MAX98357A)	4	3	4	4	3.75

El LM386 es clase AB, rinde cerca del 50 % y pide ocho componentes discretos para dar 0.5 W. El DAC I2S da el mejor audio, pero es más caro, más difícil de conseguir localmente y cambia la configuración de ALSA. El PAM8403 es un módulo armado, clase D (≈ 85 %), que entrega 1.3 W sobre $8\ \Omega$ con un filtro de tres pasivos.

3.5. Odometría

Alternativa	Téc.	Econ.	Amb.	Soc.	Total
Solo el modelo de los motores	2	5	5	3	3.40
Encoders ópticos de rueda	4	3	3	3	3.40
MPU-6050 fusionado con el modelo	4	5	5	4	4.40

Estimar la posición solo con el tiempo y el ciclo de trabajo deja derivar el rumbo en cuanto las dos llantas giran distinto, y el rumbo es lo que más deforma el mapa. Los encoders miden bien la distancia, pero había que comprarlos y montarlos en motores que no traen eje para ello. El MPU-6050 ya estaba en el inventario: su giroscopio da el rumbo y su acelerómetro la dinámica de la velocidad, que además habilita el tiempo antes de chocar.

3.6. Otras decisiones

Decisión	Elegida	Alternativas y motivo
Aislamiento	Cinco PC817	El DRV8871 acepta 3.3 V pero comparte tierra: no aísla y se descartó. El ADuM1401 aísla cuatro canales en un chip, pero es más caro y difícil de conseguir.
Acceso a GPIO	<code>pigpio</code>	<code>libgpiod</code> y <code>WiringPi</code> no generan PWM ni pulsos de servo con temporización estable; <code>pigpio</code> lo hace por DMA, mide el eco en microsegundos y da el bus I2C por el mismo demonio.
Servidor web	<code>libmicrohttpd</code>	<code>nginx</code> o <code>lighttpd</code> son un proceso aparte con CGI que mantener; un servidor en Python suma unos 40 MB al rootfs.
Decodificador MP3	<code>mpg123</code>	<code>GStreamer</code> trae decenas de MB de plugins para el mismo resultado.
Imagen base	<code>core-image-minimal</code>	<code>core-image-base</code> y <code>full-cmdline</code> arrastran utilidades y servicios que el robot no usa.
Chasis	Circular	El rectangular es más fácil de cortar, pero se traba en las esquinas al girar; el circular rota sobre su eje sin barrer área extra.
Algoritmo de cobertura	Aleatorio con rebote	Espiral y zig-zag dependen de una odometría precisa; el rebote tolera el error acumulado y saca partido de las siete direcciones del radar.

4. DI3 — Diseño de la alternativa seleccionada

4.1. Arquitectura de hardware

Un pack 2S alimenta dos rieles regulados por separado. Las cinco señales que cruzan del dominio lógico al de potencia —las cuatro entradas del puente H y la del servo— lo hacen por optoacopladores, y las dos tierras no se tocan en ningún punto.

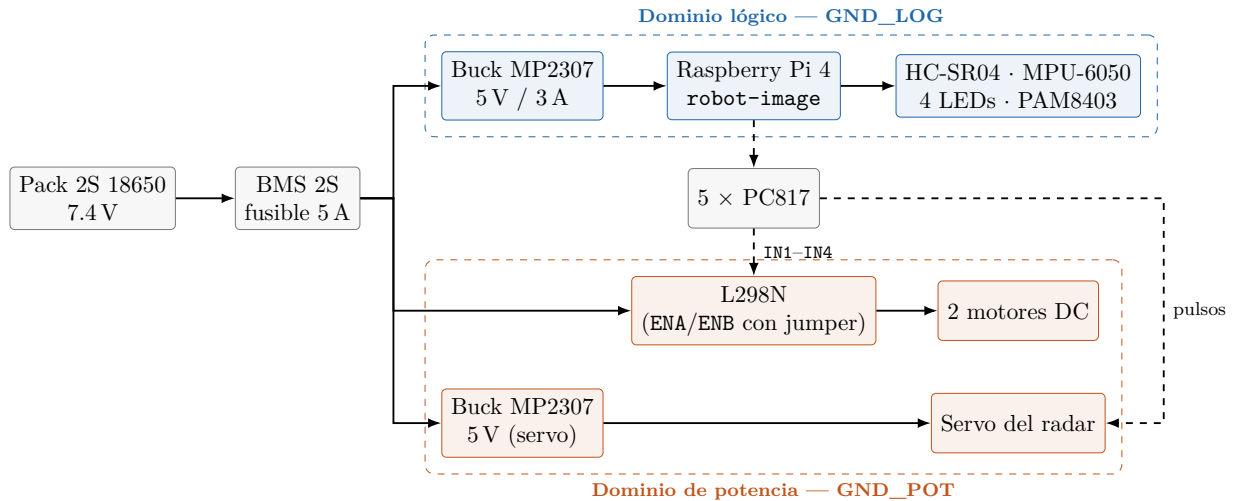


Figura 1: Arquitectura de hardware. Las líneas punteadas son señales que cruzan la barrera óptica. Los esquemas eléctricos completos están en `hardware-diagramas.pdf`.

Alimentación. Dos celdas 18650 en serie con BMS de balanceo y un fusible de 5 A. El riel lógico sale de un reductor MP2307 ajustado a 5.0 V (3 A, ≈ 93 % de eficiencia) y alimenta la Raspberry Pi por USB-C, el sensor, el MPU-6050, los LEDs y el amplificador: 2.0 A típicos y 3.0 A de pico. El riel de potencia lleva los 7.4 V directo al L298N. El servo tiene su propio reductor de 5 V en el dominio de potencia, porque sus picos de 700 mA no caben ni en el riel lógico ni en el regulador interno del L298N. Autonomía estimada: 60 minutos. Detalle en `docs/hardware-alimentacion.md`.

Aislamiento. Un PC817 por señal. Resistencia de entrada de 390Ω , que fija 5 mA por canal: con cuatro canales son 20 mA, y junto con los LEDs y el servo el conector entrega 39 mA de los 50 mA que admite. El L298N lleva los jumpers ENA y ENB puestos y se controla solo con IN1–IN4, lo que ahorra dos optoacopladores y 10 mA. Detalle en `docs/hardware-aislamiento.md`.

Sensado. Un HC-SR04 sobre un servo de 180° en el borde frontal, con el eje 10 cm por delante del centro. El pin ECHO entrega 5 V y pasa por un divisor de $1\text{ k}\Omega$ y $2\text{ k}\Omega$ (3.33 V) antes del GPIO. El MPU-6050 va entre las dos llantas, alimentado a 3.3 V para que sus resistencias de pull-up nunca pongan 5 V en el bus I2C. Detalle en `docs/hardware-sensores.md`.

Indicadores y audio. Cuatro LEDs de colores distintos a 5 mA, con la resistencia calculada para cada color. El audio sale por PWM en el GPIO 18 (overlay `audremap`), pasa por un filtro RC con corte en 19 kHz y entra al PAM8403, alimentado del riel lógico para que el ruido de los motores no llegue al parlante.

Modelo físico. Chasis circular de 22–25 cm en dos niveles, con la batería al centro y abajo para bajar el centro de gravedad, el eje de tracción ligeramente adelante del centro y una rueda loca atrás. Detalle en `docs/hardware-chasis.md`.

4.2. Arquitectura de software

Sistema operativo. La imagen `robot-image` parte de `core-image-minimal` y agrega solo lo que el robot usa; cada paquete está justificado en `docs/paquetes.md`. La capa propia `meta-robot` aporta las recetas de `librobot`, `robot-server`, `pigpio`, la configuración de ALSA y de WiFi, los

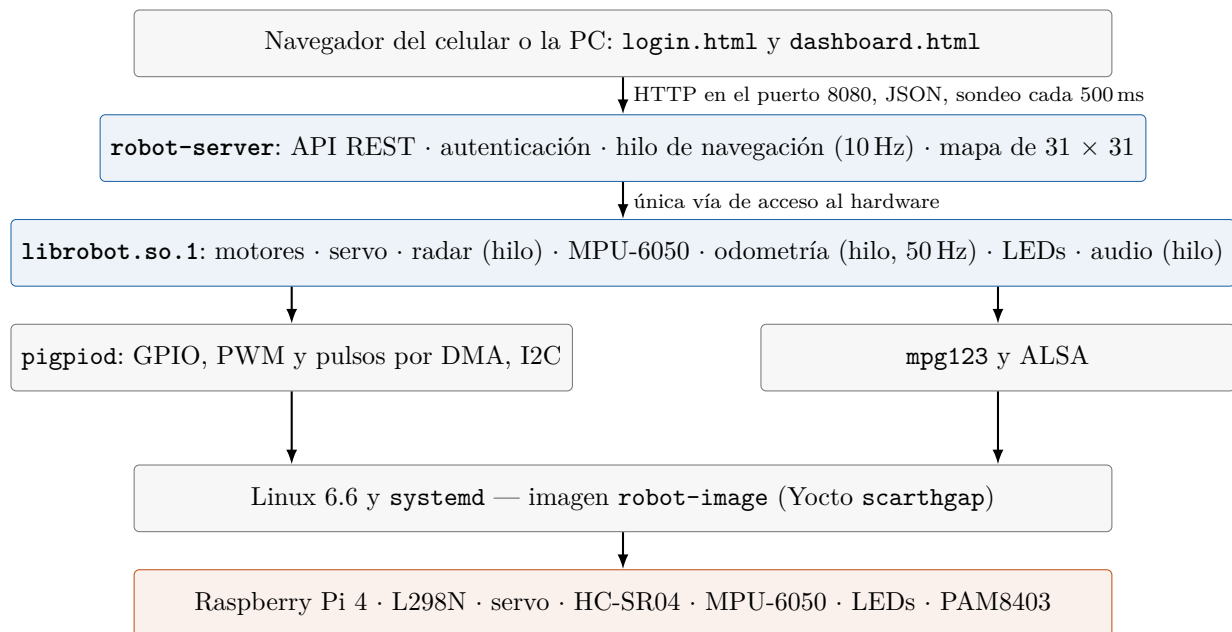


Figura 2: Arquitectura de software por capas.

ajustes de `config.txt` y la tabla de particiones. La música va en una tercera partición, de modo que ninguna supera los 200 MB.

Biblioteca. `librobot` es la única vía de acceso al hardware. Una llamada, `robot_init()`, abre la sesión e inicia dos hilos propios: la odometría a 50 Hz y el barrido del radar. El audio corre en un tercer hilo. La referencia completa está en `docs/api-librobot.md`.

Servidor. `robot-server` sirve la interfaz, expone la API y corre el hilo de navegación a 10 Hz. No espera al radar: lee la última lectura de cada ángulo cuando la necesita. Las contraseñas se guardan como resumen SHA-256 y la sesión viaja en una cookie `HttpOnly` que caduca a los 10 minutos de inactividad.

Arranque. Dos unidades `systemd`, habilitadas desde las recetas. `robot-server` declara `Requires=pigpiod.service` y `Restart=on-failure` con un límite de cinco reintentos por minuto.

4.3. Navegación

El robot avanza hasta que detecta un obstáculo por cualquiera de dos vías: el tiempo antes de chocar baja de 1.2 s, o alguna lectura del cono frontal (60°, 90°, 120°) baja de 20 cm. Entonces se detiene, suena el aviso y enciende el LED rojo. En modo autónomo sigue la evasión:

1. retrocede medio segundo;
2. espera un barrido completo del radar, ya quieto;
3. elige el rumbo con más espacio entre las seis direcciones que no son el frente; las que quedan a menos de 25 cm de la mejor compiten al azar, que es la parte aleatoria del rebote;
4. gira en lazo cerrado sobre el rumbo del giroscopio hasta cubrir el ángulo elegido, no durante un tiempo fijo.

En modo manual la misma detección actúa como freno de seguridad.

4.4. Elementos propios del diseño

- **Tiempo antes de chocar.** Cada lectura frontal se congela con el avance acumulado en ese instante. Entre dos lecturas, la distancia se proyecta con lo que el robot avanzó y se divide entre la velocidad actual: $t = (d - \Delta_{\text{avance}})/v$. La cuenta regresiva sigue bajando aunque el sensor esté mirando a un lado, y compensa la baja frecuencia de la lectura frontal del radar.
- **Fusión de la velocidad.** Un filtro complementario deja pasar la dinámica rápida del acelerómetro y toma del modelo de los motores la tendencia lenta ($\tau = 1$ s); con los motores detenidos la velocidad se fuerza a cero. Así el sesgo del acelerómetro no se acumula.
- **El canal del servo no invierte.** Los cuatro canales del puente H van en emisor común, que invierte la señal, y la inversión se compensa en un único punto de `lib_motors.c`. Para el servo eso no sirve: en reposo el GPIO queda en bajo, y un canal inversor le entregaría un nivel alto permanente que lo llevaría contra su tope. Su canal se armó en seguidor de emisor, que no invierte.
- **El reposo es seguro.** Con los GPIO en bajo, las cuatro entradas del L298N quedan en alto por sus resistencias: con el puente habilitado eso es freno, no movimiento.
- **Desarrollo sin hardware.** Un simulador reemplaza a `pigpiod` por un robot virtual en una sala de 4×4 m, sin tocar una línea de la biblioteca. Una máquina `qemuarm64-robot` con el mismo *tune* Cortex-A72 arranca la imagen completa con los mismos binarios de espacio de usuario que corren en la Pi.
- **Imagen reproducible sin secretos.** Si falta el archivo de credenciales de WiFi, la receta usa la plantilla y avisa: un clon limpio produce la imagen sin pasos manuales, como exige R18.

4.5. Cómo responde el diseño a seguridad, costo e impacto

Aspecto	Respuesta del diseño
Seguridad	BMS, fusible, dos rieles, aislamiento óptico con tierras separadas, divisor en ECHO, MPU-6050 a 3.3 V, freno en reposo y freno automático en modo manual, <code>pigpiod</code> con -1 (solo interfaz local), inicio de sesión obligatorio.
Costo	Se reutilizó el sensor, el MPU-6050 y el L298N del grupo. El radar evitó comprar dos sensores; el MPU-6050 evitó los encoders; el PAM8403 reemplazó ocho componentes discretos.
Ambiental	Reguladores conmutados y amplificador clase D; módulos con conectores desconectables y reutilizables; celdas recargables y desmontables.

5. DI4 — Validación del diseño final

5.1. Estrategia

La validación se hizo en tres niveles, de menor a mayor costo de cada iteración:

1. **Simulador en el host.** `sim/prueba_librobot` ejercita cada módulo de la biblioteca contra el robot virtual y compara con la verdad del mundo simulado: 47 comprobaciones.

2. **Emulación ARM.** El mismo programa, compilado con el Toolchain-SDK, corre bajo `qemu-aarch64`; y la imagen completa arranca en la máquina `qemuarm64-robot`.
3. **Target.** La imagen en la Raspberry Pi 4 con el robot armado, siguiendo los procedimientos de verificación de `docs/`.

5.2. Matriz de cumplimiento

Id.	Cómo se validó	Resultado	Evidencia
R1	Dos minutos en modo autónomo sobre el simulador: 18 evasiones y ningún choque. Prueba de campo en un área delimitada.	Cumple	<code>docs/navegacion-radar.md</code>
R2	Barrido completo contra el mundo simulado: los siete ángulos a menos de 2 cm de la distancia real, a 5 lecturas por segundo.	Cumple	<code>sim/prueba_librobot.c</code>
R3	Avance, retroceso, giros y saturación verificados en el simulador; sentido de giro confirmado con el robot elevado.	Cumple	<code>docs/evidencias/ejecucion-cruzada-qemu.md</code>
R4	Cada LED cambia de estado al ordenarlo y sigue al modo activo.	Cumple	<code>docs/evidencias/servidor-web.md</code>
R5	Reproducción en un hilo propio mientras el hilo de navegación corre.	Cumple	<code>lib/lib_audio.c</code>
R6	Listar, reproducir, pausar, detener y volumen por HTTP.	Cumple	<code>sim/prueba_api.sh</code>
R7	Tono de prueba y sonido de evento por el parlante del robot.	Cumple	<code>docs/hardware-sensores.md</code>
R8	Los cuatro eventos disparan su sonido; el aviso pausa la música y la reanuda.	Cumple	<code>docs/api-librobot.md</code>
R9	Cambio de modo y movimiento manual por HTTP; los controles manuales se bloquean en autónomo.	Cumple	<code>docs/evidencias/servidor-web.md</code>
R10	<code>/api/status</code> entrega modo, sensores, LEDs, audio y mapa en una sola consulta, cada 500 ms.	Cumple	<code>docs/evidencias/servidor-web.md</code>
R11	La grilla crece en tiempo real: de 79 a 85 celdas visitadas en 6 s.	Cumple	<code>docs/evidencias/servidor-web.md</code>
R12	Credenciales inválidas y consultas sin sesión devuelven 401.	Cumple	<code>docs/evidencias/servidor-web.md</code>
R13	Tras un <code>kill -9</code> , el servicio vuelve con otro PID y <code>NRestarts=1</code> .	Cumple	<code>docs/evidencias/systemd-reinicio.md</code>
R14	El servidor declara <code>NEEDED</code> <code>librobot.so.1</code> y no toca GPIO.	Cumple	<code>docs/evidencias/binarios-target.md</code>

Id.	Cómo se validó	Resultado	Evidencia
R15	Los logs muestran <code>aarch64-poky-linux-gcc</code> ; los binarios son ELF ARM aarch64 y no ejecutan en el host.	Cumple	<code>docs/evidencias/</code>
R16	Imagen sobre <code>core-image-minimal</code> ; cada paquete agregado, listado y justificado.	Cumple	<code>docs/paquetes.md</code>
R17	Ambas recetas heredan <code>cmake</code> ; la biblioteca se empaqueta en <code>librobot</code> y <code>librobot-dev</code> .	Cumple	<code>meta-robot/</code>
R18	<code>bitbake robot-image</code> produce la imagen desde un clon limpio.	Cumple	<code>docs/yocto-setup.md</code>
R19	Ver la sección 5.4.	Ver tabla	<code>docs/metricas.md</code>

[PENDIENTE: R3: confirmar el estado final de la velocidad variable; ver el hallazgo 3 de la sección 5.6]

5.3. Validación de la seguridad

Antes de conectar la Raspberry Pi se ejecutaron, en orden, los procedimientos de cinco pasos de `docs/hardware-aislamiento.md` y `docs/hardware-alimentacion.md`. Los criterios de aceptación:

Verificación	Criterio
Continuidad entre GND_LOG y GND_POT, todo desenergizado	Circuito abierto ($> 1\text{ M}\Omega$)
Diferencia de tensión entre las dos celdas	Menor a 0.05 V
Salida del reductor de 5 V en vacío y con 2 A de carga	$5.00 \pm 0.10\text{ V}$ y $\geq 4.90\text{ V}$
Salida del divisor de ECHO al disparar el sensor	3.3 V como máximo
Entradas del L298N con los optoacopladores en reposo	$\approx 5\text{ V}$: motores frenados
Salida del canal del servo con el GPIO en bajo	$\approx 0\text{ V}$
<code>vcgencmd get_throttled</code> navegando con audio	0x0

[PENDIENTE: anotar los valores medidos en cada verificación]

5.4. Validación de la eficiencia

Métrica	Referencia	Medido	Método
Rootfs	≤ 200 MB	129 MB	du sobre el rootfs de la imagen construida. La partición raíz es de tamaño fijo, 180 MiB.
Tiempo de arranque	≤ 15 s	17 s	Instante en que <code>robot-server</code> queda activo, en la Raspberry Pi 4 con WiFi.
RAM en operación	—	[PENDIENTE: medir]	/proc/meminfo y VmRSS de cada proceso.
CPU en operación	—	[PENDIENTE: medir]	/proc/stat durante 60 s.

El rootfs queda en el 65 % del presupuesto. El arranque excede la referencia por 2 s: `robot-server` espera a que la red esté operativa (`network-online.target`), y la asociación WPA2 más la concesión DHCP dominan ese tiempo. Se considera una desviación justificada, porque sin red el panel de control no es alcanzable y el servicio no estaría operativo en el sentido del enunciado. El método completo está en `docs/metricas.md`.

5.5. Validación del costo y del impacto

- **Costo.** **[PENDIENTE: comparar el costo real con la estimación de 49–99 USD.]**
- **Impacto ambiental.** Todos los módulos quedaron con conectores desconectables; la Raspberry Pi se devuelve sin soldaduras ni adhesivos; el pack es recargable y desmontable.

5.6. Hallazgos de la validación y acciones correctivas

Validar no fue solo confirmar: varias pruebas cambiaron el diseño.

	Hallazgo	Causa	Acción
1	El límite de reintentos de <code>systemd</code> no existía.	<code>StartLimitIntervalSec</code> estaba en <code>[Service]</code> ; desde <code>systemd 229</code> va en <code>[Unit]</code> y en otro lugar se ignora en silencio.	Se movió a <code>[Unit]</code> . Lo detectó <code>systemd-analyze verify</code> .
2	Con los optoacopladores, los motores girarían al revés.	El PC817 en emisor común invierte la señal.	Compensación en un único punto de <code>lib_motors.c</code> ; el simulador invierte igual que la placa, así que quitarla hace fallar la prueba.
3	Con PWM de 1 kHz los motores no se movieron.	El PC817 con pull-up de 4.7 k Ω tarda decenas de microsegundos en apagarse; sumado a los 2 V que cae el L298N, a ciclos medios el motor no alcanza su tensión de arranque.	[PENDIENTE: describir la corrección aplicada: PWM a 100 Hz, pull-up de 1 kΩ o velocidad fija]

	Hallazgo	Causa	Acción
4	El servidor arrancaba a los 130 s.	<code>eth0</code> sin cable y dos clientes DHCP sobre <code>wlan0</code> agotaban la espera de red.	Ethernet marcada como no requerida y un solo cliente DHCP: 17 s.
5	El vaivén del servo sacudía el sensor y el chasis.	El servo iba a su velocidad máxima entre paradas.	Rampa a un tercio de la velocidad, con un pulso intermedio cada 20 ms.
6	El <i>build</i> colgaba el host al 67 %.	Tres compilaciones pesadas a la vez agotaron 7 GB de RAM y el swap.	Dos hilos y regulación por presión de recursos (<code>BB_PRESSURE_MAX_MEMORY</code>).

5.7. Limitaciones conocidas

- El radar usa un solo sensor físico para cubrir las detecciones frontal y lateral. El diseño deja libres los GPIO 22 y 10 para un segundo HC-SR04 fijo al frente.
- Sin encoders, la distancia recorrida depende de la calibración del modelo de los motores; el mapa sirve para la sesión, no como posición absoluta.
- Una lectura sin eco no significa camino libre: una pared oblicua desvía el pulso. El mapa no usa esas lecturas para despejar celdas.
- Los requerimientos opcionales no se implementaron.

6. Conclusiones

El diseño cumple los requerimientos obligatorios con un rootfs de 129 MB y un modelo físico construido con el inventario del grupo y una compra acotada. Las decisiones que más pesaron —radar sobre servo, pack 2S con dos rieles, MPU-6050 fusionado con el modelo de los motores— se tomaron comparando alternativas con criterios técnicos, económicos, ambientales y sociales, y cada una dejó una debilidad conocida que el propio diseño atiende.

La validación en tres niveles permitió avanzar el software antes de tener el hardware, pero también mostró su límite: el problema de la PWM a través del optoacoplador solo apareció en la placa real. El diseño de un sistema embebido se cierra en el hardware.

7. Referencias

- J. González Gómez, *Especificación Proyecto I*, CE-1113 Sistemas Empotrados, Tecnológico de Costa Rica, 2026.
- Yocto Project, *Reference Manual y Development Tasks Manual*, versión 5.0 (scarthgap).
- Raspberry Pi Ltd., documentación de la Raspberry Pi 4 Model B: GPIO, `config.txt` y *device tree overlays*.
- `pigpio`, documentación de la interfaz `pigpiod_if2`.
- Hojas de datos: Sharp PC817, STMicroelectronics L298N, InvenSense MPU-6050, Diodes Inc. PAM8403, Monolithic Power MP2307, HC-SR04.
- Repositorio del proyecto: <https://github.com/Randall-BL/EmpotradosProyecto1>.